Today’s agenda

- Assignment07 presentation
- FreeRTOS recap
- Queues recap
- Deadlocks / queue topology
- Introduction to Assigment08
- Lab09

SDU

Course outline

- **The plan is indicative, subject to change!**

Date	Lecture	Subject(s)	Lab	Assignment due
Feb 2	1	Introduction; ARM Cortex-M4	Lab1: Setting up the development env.	
Feb 9	2	EMP coding standard; Bit manipulation	Lab2: Bit manipulation	
Feb 16	3	State machines; The compiler chain; EMP-Board	Lab3: State machines	PF1 – Assignment 1
Feb 23	4	The task model; The pre-processor	Lab4: A clock radio task model	PF1 – Assignment 2
Mar 2	5	Queues and semaphores; Debugging	Lab5: Debug with a serial connection	PF1 – Assignment 3
Mar 9	6	Run to complete scheduler	Lab6: RTCS, Run to compile scheduler	PF1 – Assignment 4
Mar 16	7	More debugging; C: printf()	Lab7: RTCS, Debugger	PF1 – Assignment 5
March 23	8	FreeRTOS	Lab8: FreeRTOS	PF1 – Assignment 6
March 30 April 6		Easter holiday		
Apr 13	9	Re-entrance, more queues	Lab9: FreeRTOS (continued)	PF1 – Assignment 7
Apr 20	10	Assembler in C,	Work on the final assignment	PF1 – Assignment 8
Apr 27	11	Work on the final assignment, consultations		
May 4	12	Work on the final assignment, consultations		
May 11	13	Poster session		PF2- Final Assignment

SDU

Notes about final assignment

- Work needs to be done WITHIN groups
- Work cannot be done between groups
- Code cannot be taken from any other student groups/years
- If code is taken from some online solution, this needs to be discussed with teacher beforehand and mentioned in the report
- Absolutely no ChatGPT, Claude, or other generative AI solutions
- If any of the above points are not adhered to, it will be treated as plagiarism

SDU

Recap - FreeRTOS

- RTOS
  - Real-time operating system
  - Responds to events within a strictly defined time (deadline)
  - Processes data as it comes in without delay
  - Scheduler is deterministic
  - Used when
    - Meeting deadlines is critical for task completion
    - Results only make sense if delivered within time constraints

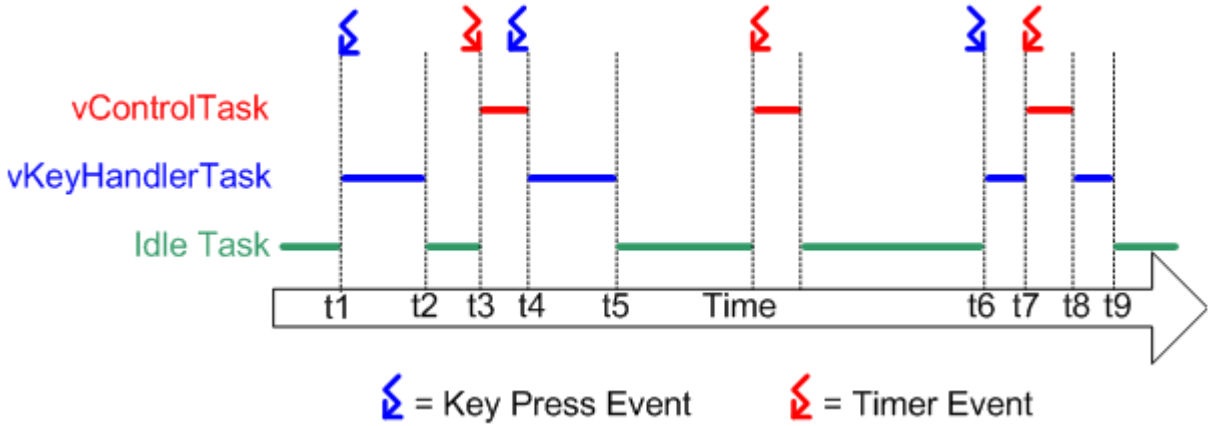
SDU

Recap - FreeRTOS

- FreeRTOS
  - Small-enough for microcontrollers
  - Provides only
    - Real-time scheduling
    - Inter-task communication
    - Timing and synchronization primitives
  - More a real-time kernel than OS

SDU

Recap - FreeRTOS



- Scheduler
  - Part of kernel deciding on task execution
  - Multi-tasking – executing tasks seemingly simultaneously
  - Pre-emptive scheduling
    - Runs tasks with highest priority first
    - Tasks of identical priority share CPU time (round robin time slicing)
  - Task states: running, ready, blocked, suspended

SDU

Recap - FreeRTOS

- Additional features
  - Message queues
  - Binary and counting semaphores
  - Mutexes
  - Context switching
    - When kernel switches tasks, it saves context (registers, stack, etc.)
    - When the task is resumed the context is restored

SDU

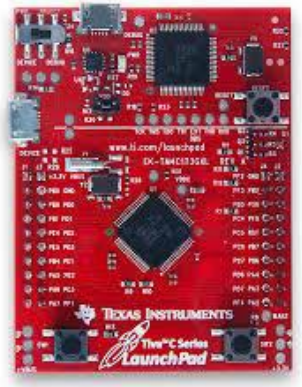
Recap – FreeRTOS online resources

- freertos.org -> resources
- https://www.freertos.org/Documentation/RTOS_book.html
- Download FreeRTOS for examples!
- More info on memory management https://www.freertos.org/a00111.html

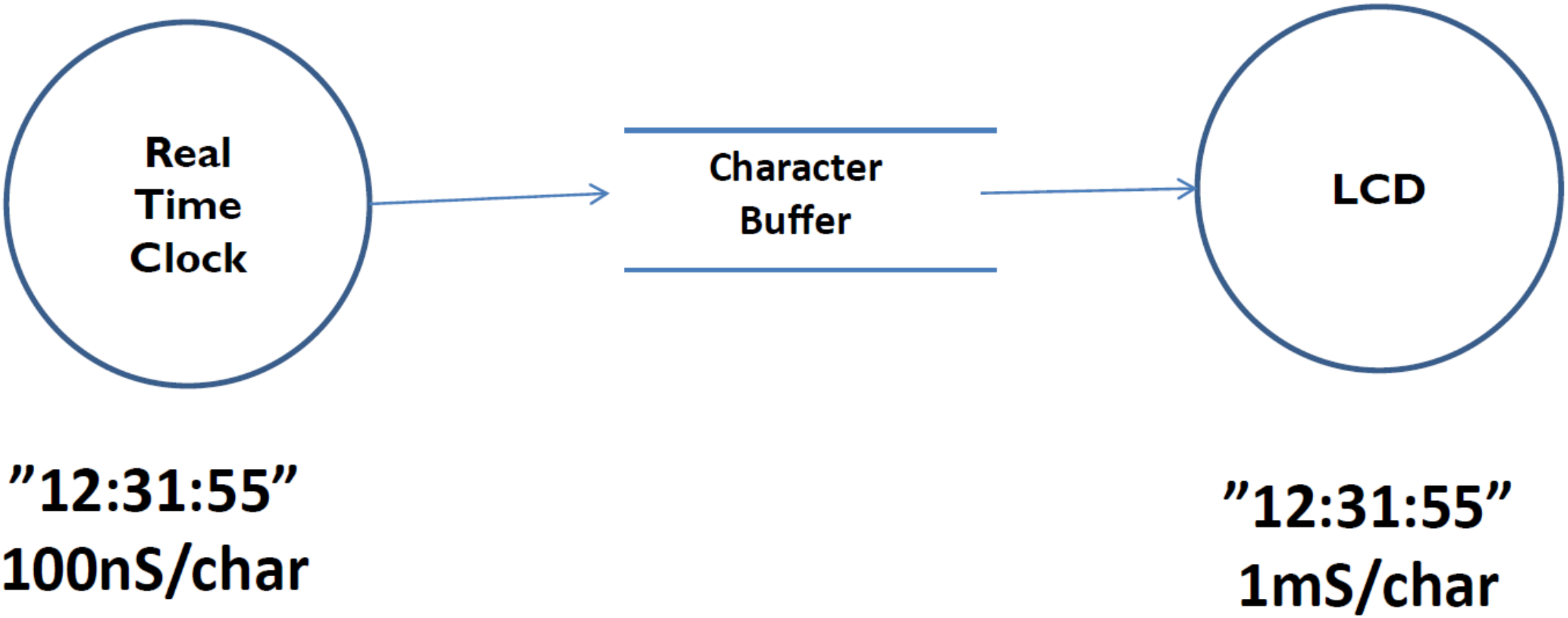
SDU

SDU

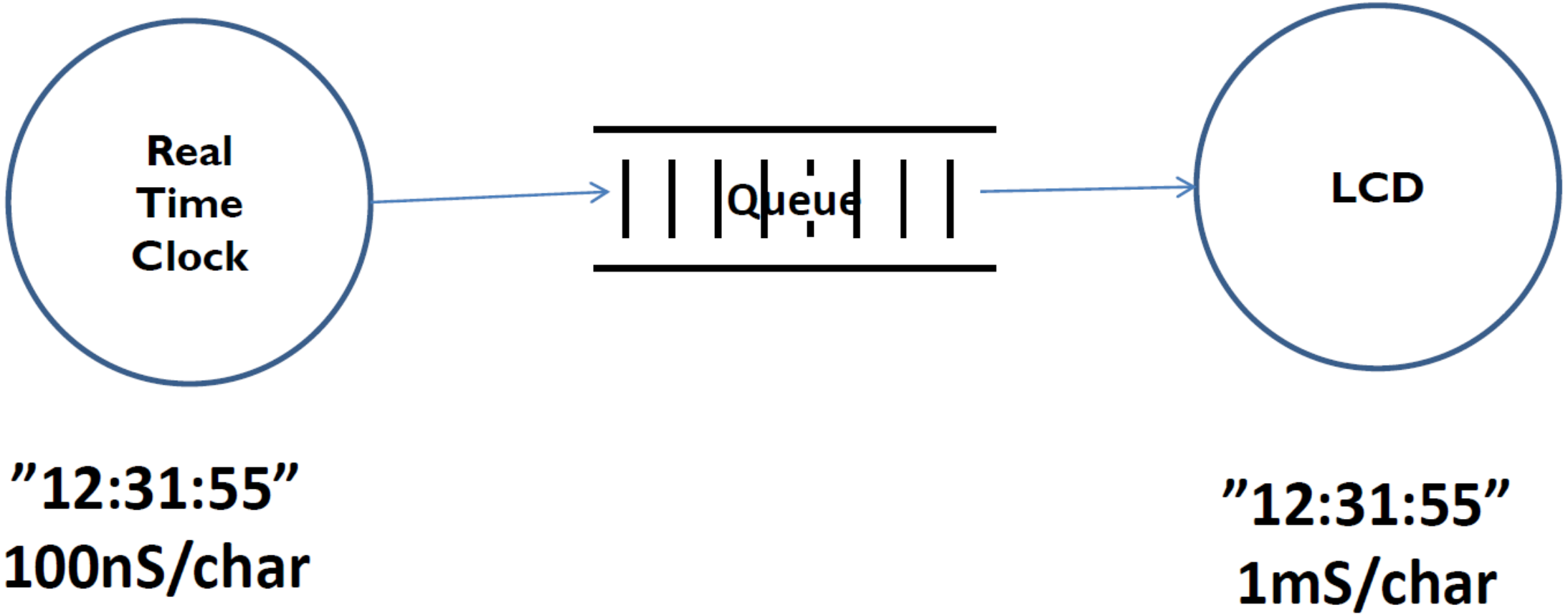
Recap of queues & mutexes
Deadlocks, queues continued



Recap: producer – consumer model



Recap: producer – consumer model



Recap - a queue object

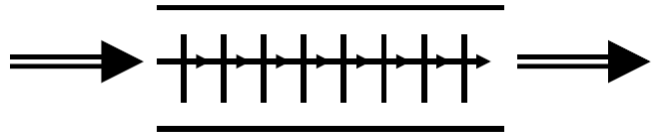
A queue is…
…an individual object in the task diagram
…has a unique name, number, id, key or handle
…implement a FIFO strategy
…offer an API for:

- creating (Create, Open, …)
- insert data (Put, Write, Send, Post, …)
- fetch data (Get, Read, Receive, Retrieve, …)
- show the state of the queue (number of items in the queue) (Full, Empty, Length, …)
- Maybe: read data without removing it from the queue (Peek, …)

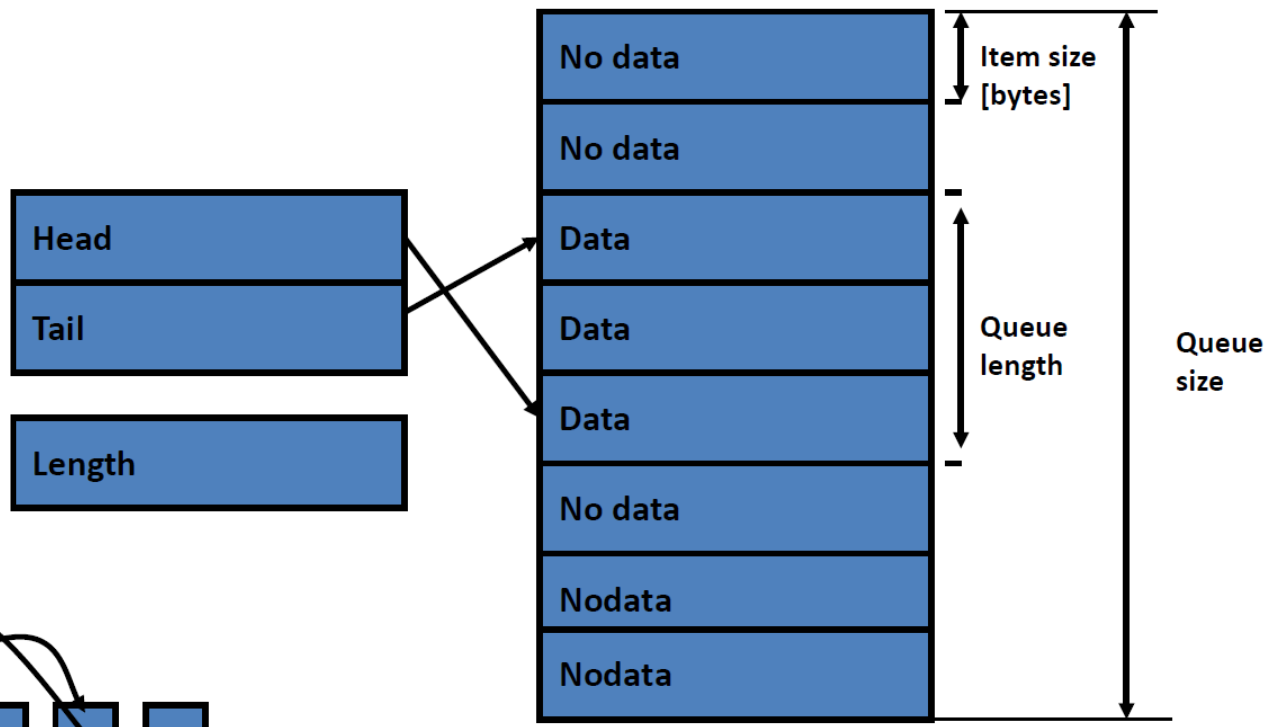
SDU

Recap: Queue implementation

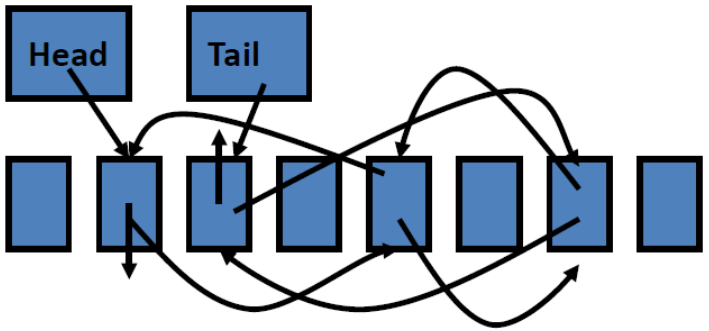
Move data through the queue (like supermarket)



Use pointers (pull a number
-like the post office)



Linked List (like exam
– call the next)



SDU

Recap: queue.c

```
typedef struct
{
            INT8U      head;
            INT8U      tail;
            INT8U      len;
            INT8U buf[QUEUE_SIZE];
} queue_descriptor;
```

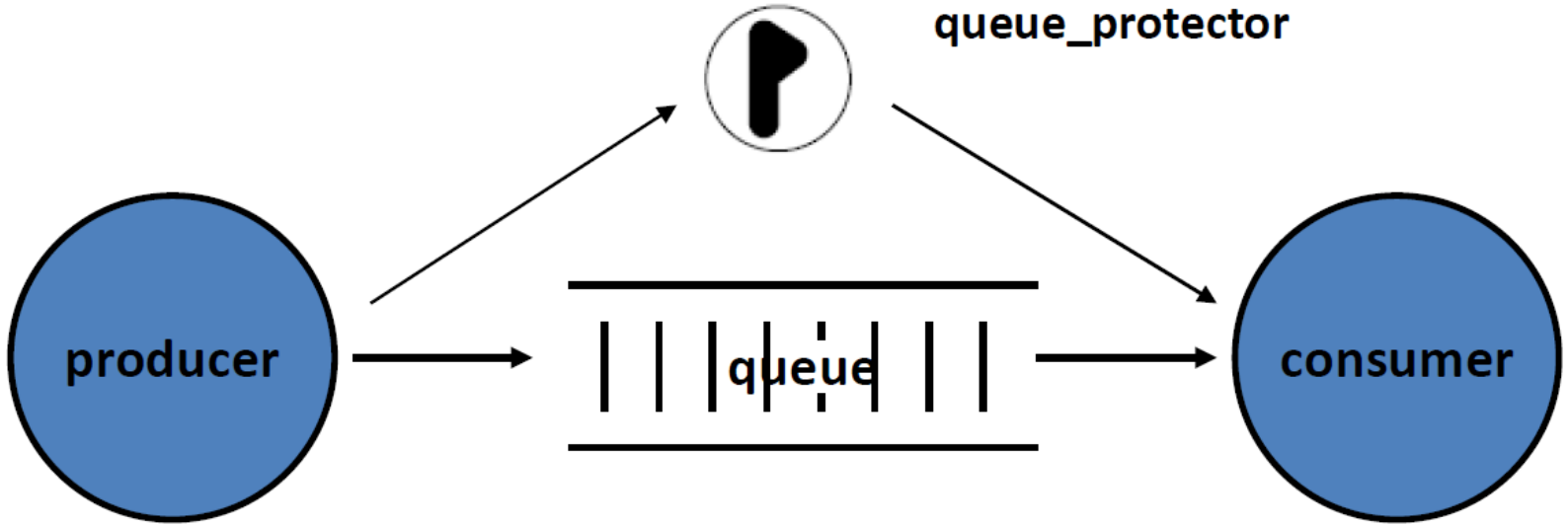
```
queue_descriptor queue_pool[NOF_QUEUES];
```

```
INT8U queue_put( id, ch )
INT8U id;
INT8U ch;
/************************************************
*   Function : See module specification (.h-file).
************************************************/
{
  INT8U result = FALSE;

  if( id < NOF_QUEUES )
  {
    if( queue_pool[id].len < QUEUE_SIZE )
    {
      queue_pool[id].buf[queue_pool[id].tail++] = ch;
      if( queue_pool[id].tail >= QUEUE_SIZE )
        queue_pool[id].tail = 0;
      queue_pool[id].len++;
      result = TRUE;
    }
  }
  return( result );
}
```

```
INT8U queue_get( id )
INT8U id;
/************************************************
*   Function : See module specification (.h-file).
************************************************/
{
  INT8U Ch = '\0';

  if( id < NOF_QUEUES )
  {
    if( queue_pool[id].len )
    {
      Ch = queue_pool[id].buf[queue_pool[id].head++];
      if( queue_pool[id].head >= QUEUE_SIZE )
        queue_pool[id].head = 0;
      queue_pool[id].len--;
    }
  }
  return( Ch );
}
```

Protection of queues

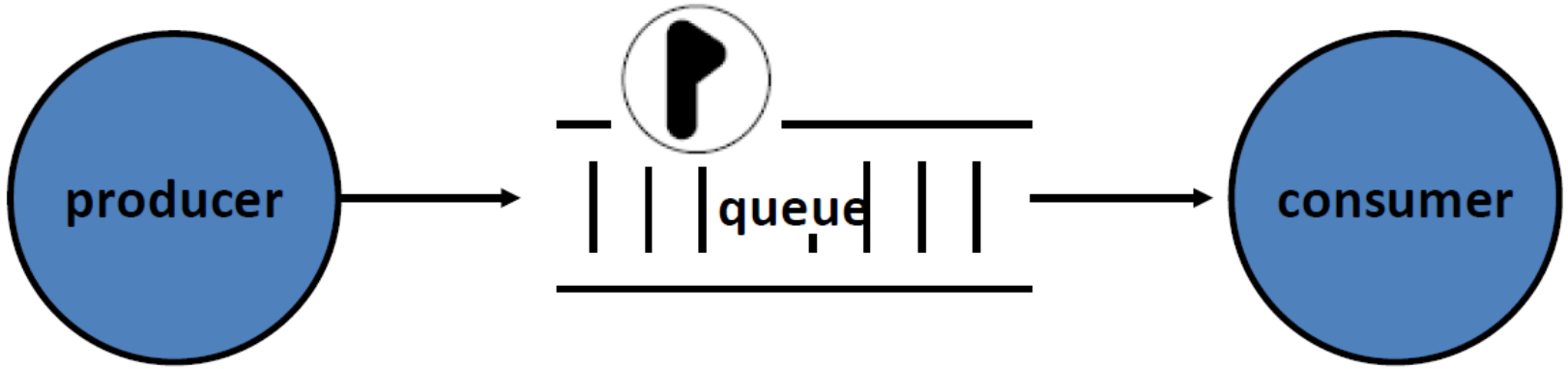
A queue is a shared resource, and as such, it must be protected by a mutex semaphore.



SDU

Protection of queues

- Because of that, the mutex is often build into the queue or closely connected to the queue.
- As we are now using pre-emptive scheduling protection of queues is crucial



SDU

Recap: Semaphore - mutex

- Secures mutually exclusive blocks to prevent concurrent access to the same resource
- Binary semaphore
- Creates critical sections

Task A – clock update

```
mutex.wait()
        #critical section
        update(clock)
mutex.signal()
```

Task B – clock display

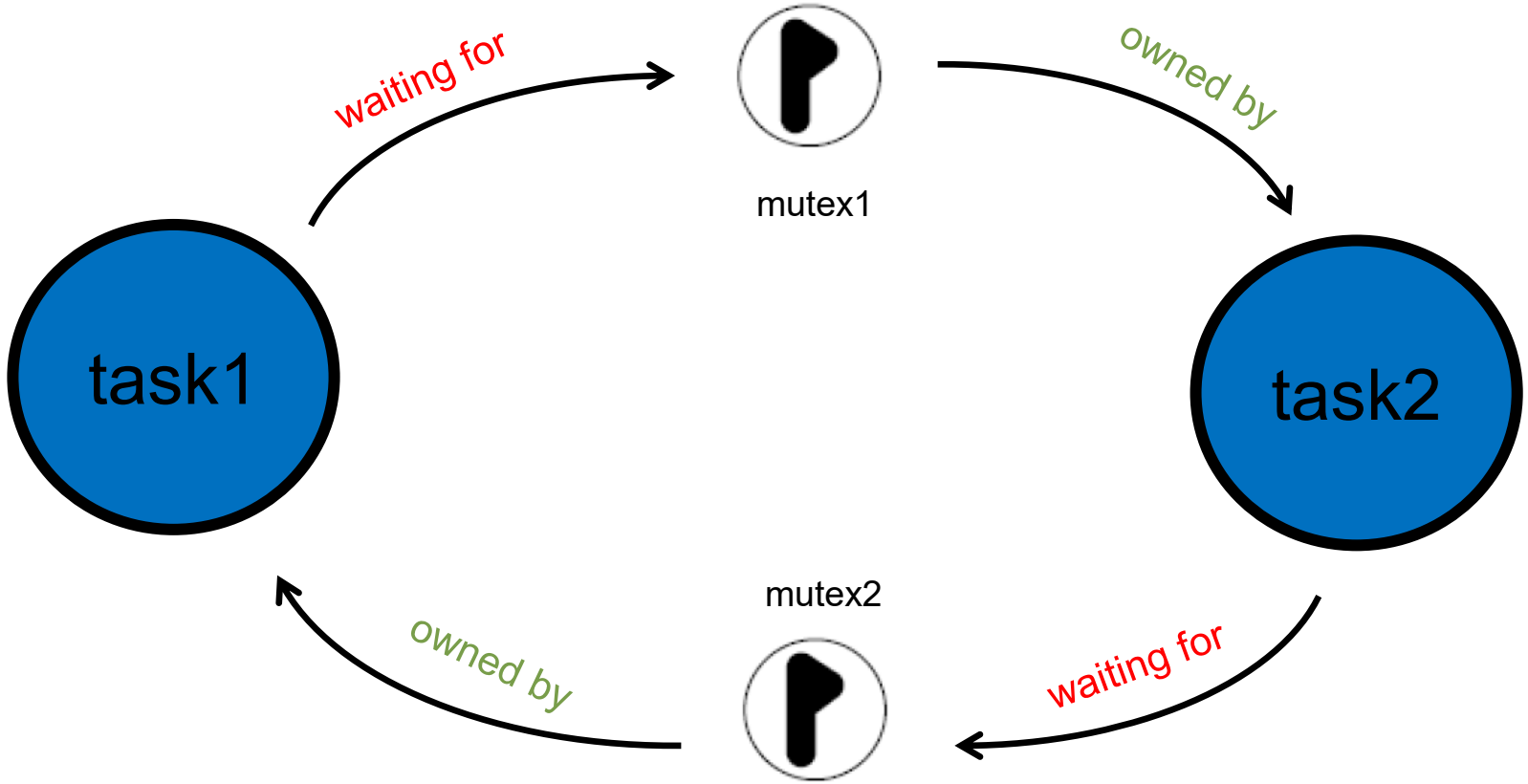
```
mutex.wait()
        #critical section
        display(clock)
mutex.signal()
```

- Example – contentious meetings with duckie

SDU

Beware of deadlocks

- Deadlocks appear when two or more tasks are waiting at each other to continue their execution
- This usually happens through resources being blocked with mutexes

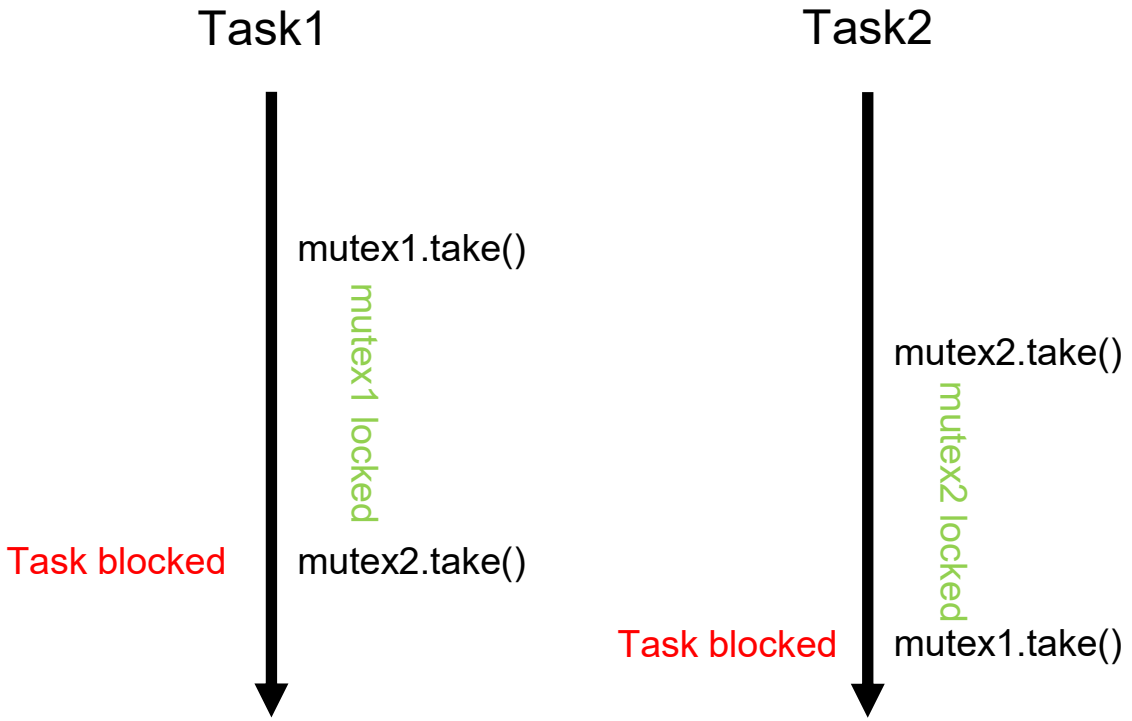


SDU

Deadlock in real-life

Deadlock in other words

- A process cannot proceed because it needs to obtain a resource held by another process but it itself is holding a resource that the other process needs

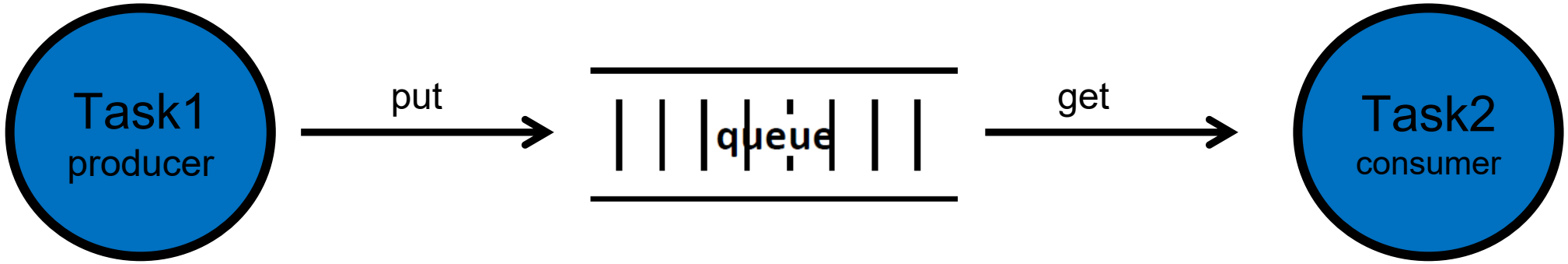


Coffman’s 4 conditions of deadlock

- **Mutual exclusion** – a resource can be held by at most one process
- **Hold and wait -** processes that already hold a resource can wait for another resource
- **Non-preemption** – a resource, once granted, cannot be taken away
- **Circular wait** – two or more processes are waiting for resources held by one of the other processes

SDU

Blocking / non-blocking

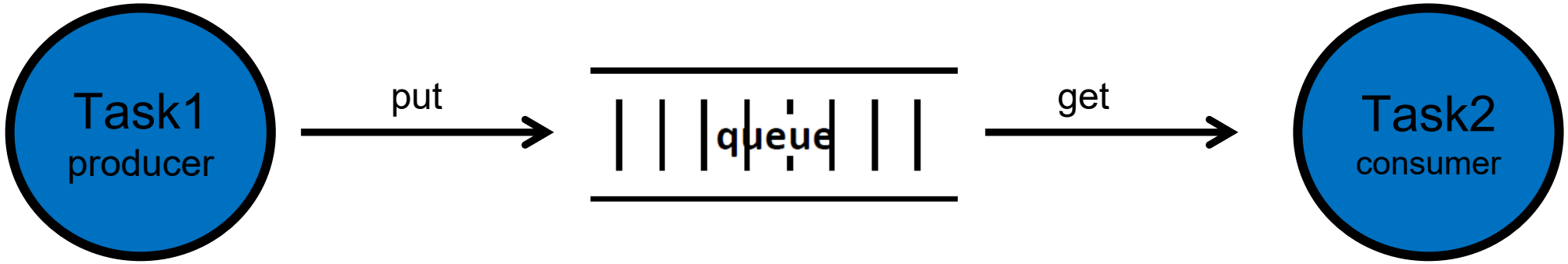


- Producer – put()
- If the queue is already full:
  - Throw away the data
  - Do something else and try later
  - Overwrite old data
  - Wait (block)

- Consumer get()
- If the queue is already empty:
  - Do something else and try later
  - Wait (block)

SDU

Blocking

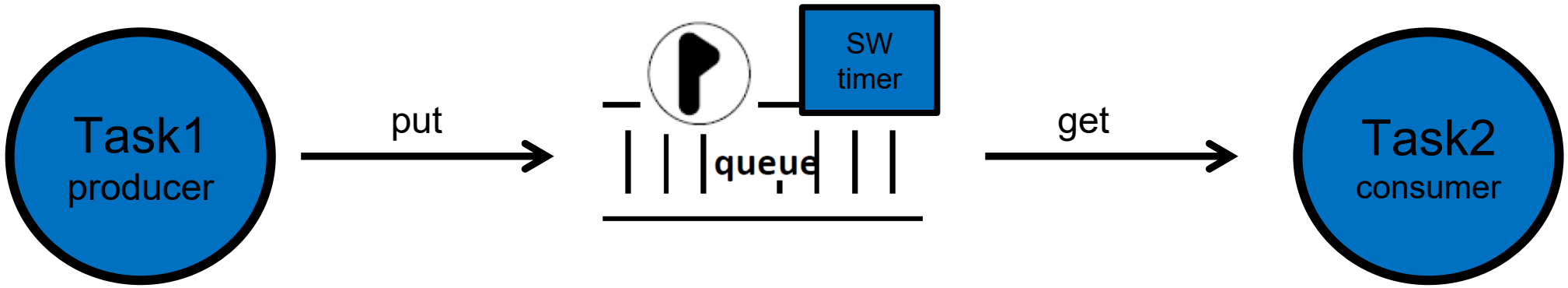


- Producer – put()
- Blocking:
  - Forever
  - Timeout

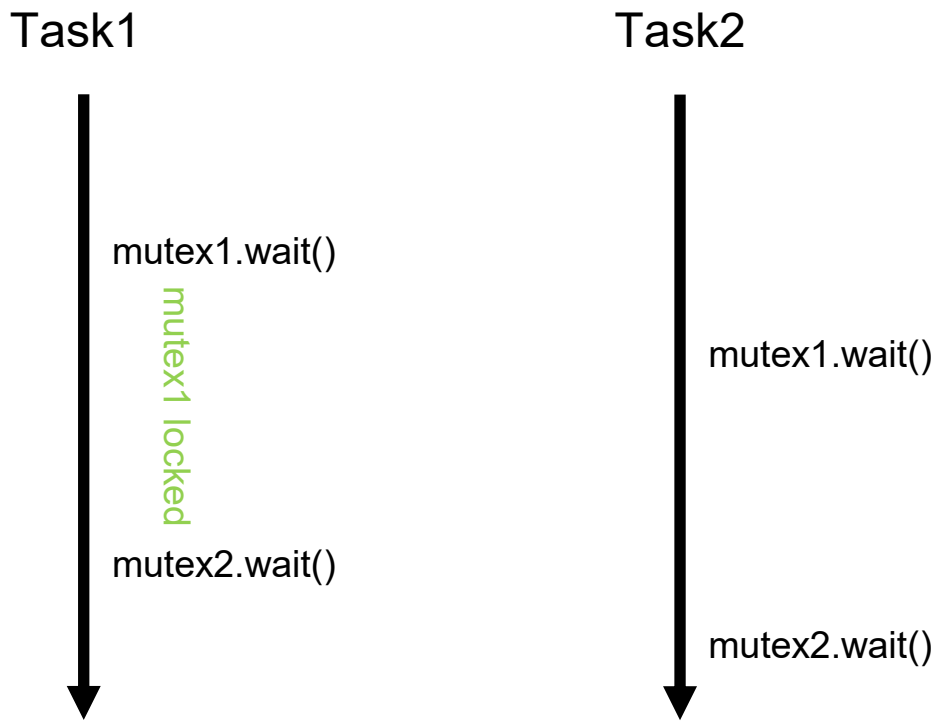
- Consumer get()
- Blocking:
  - Forever
  - Timeout

SDU

How to resolve? Timeout



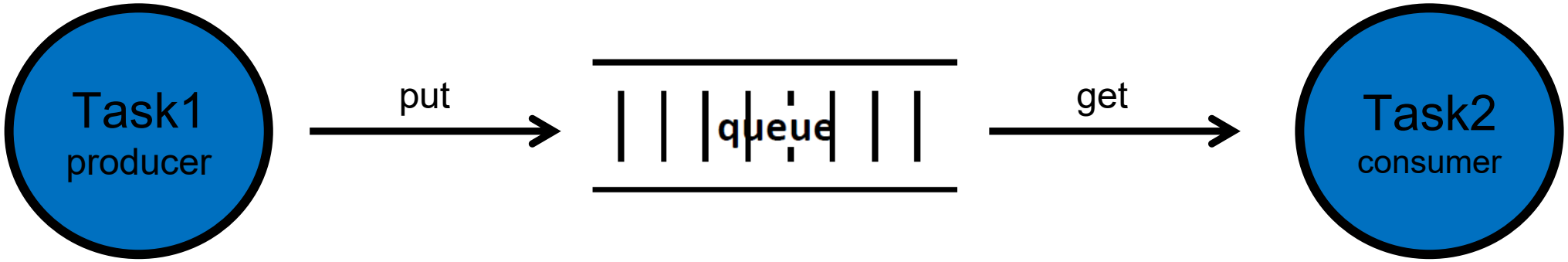
How to resolve? Ordering



SDU

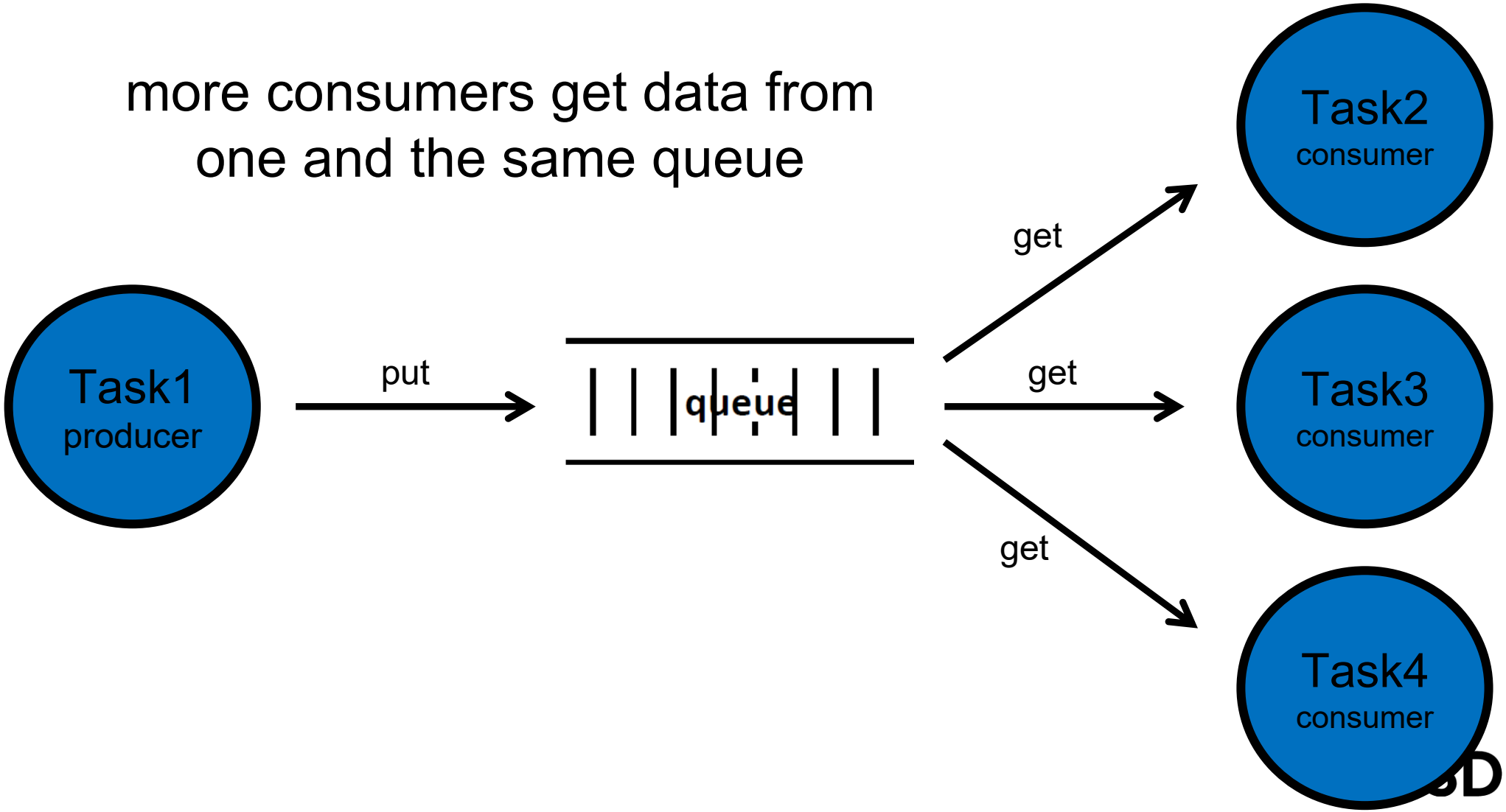
Queue topology

one producer – one consumer



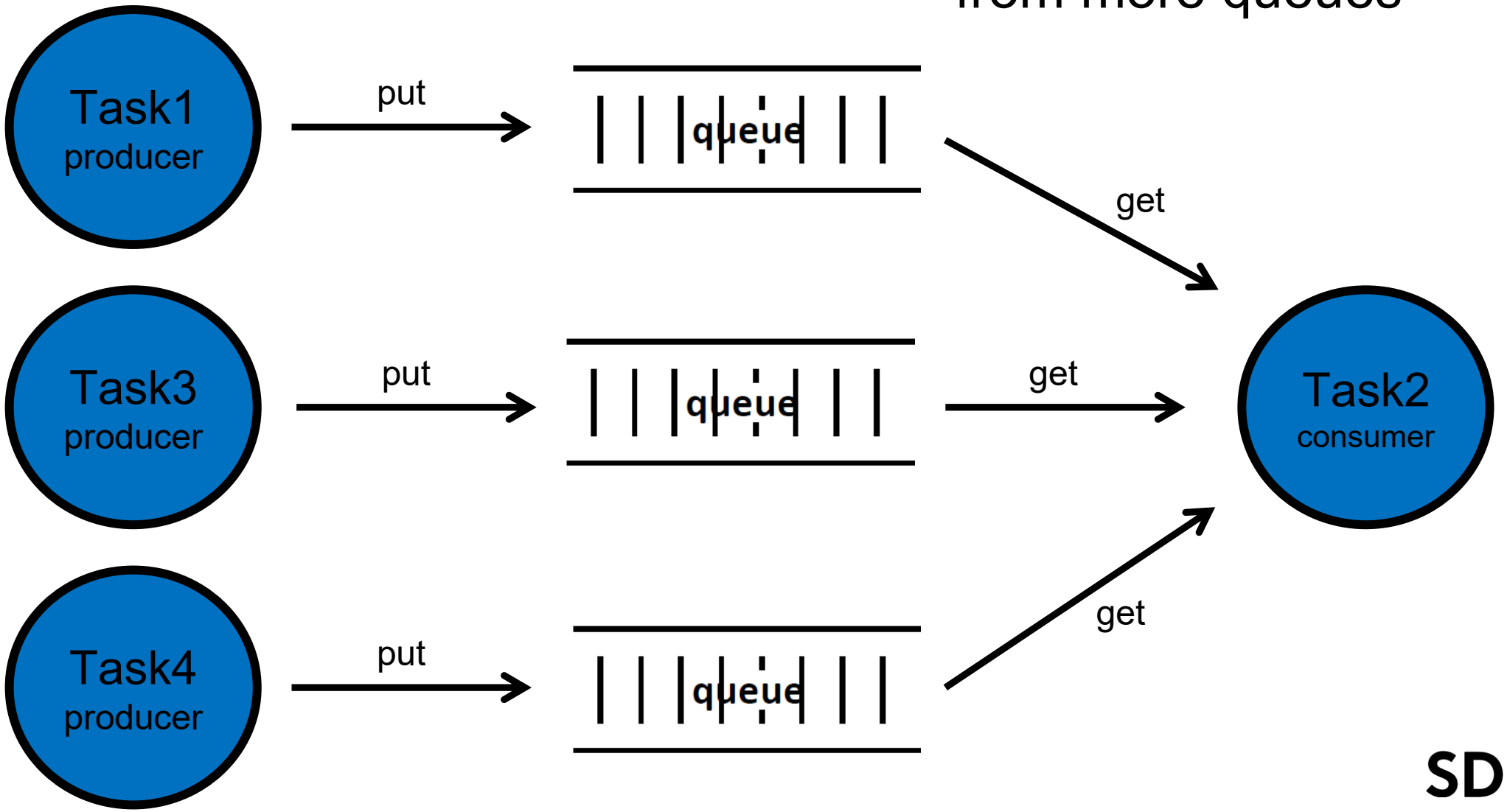
Queue topology

more consumers get data from one and the same queue



Queue topology

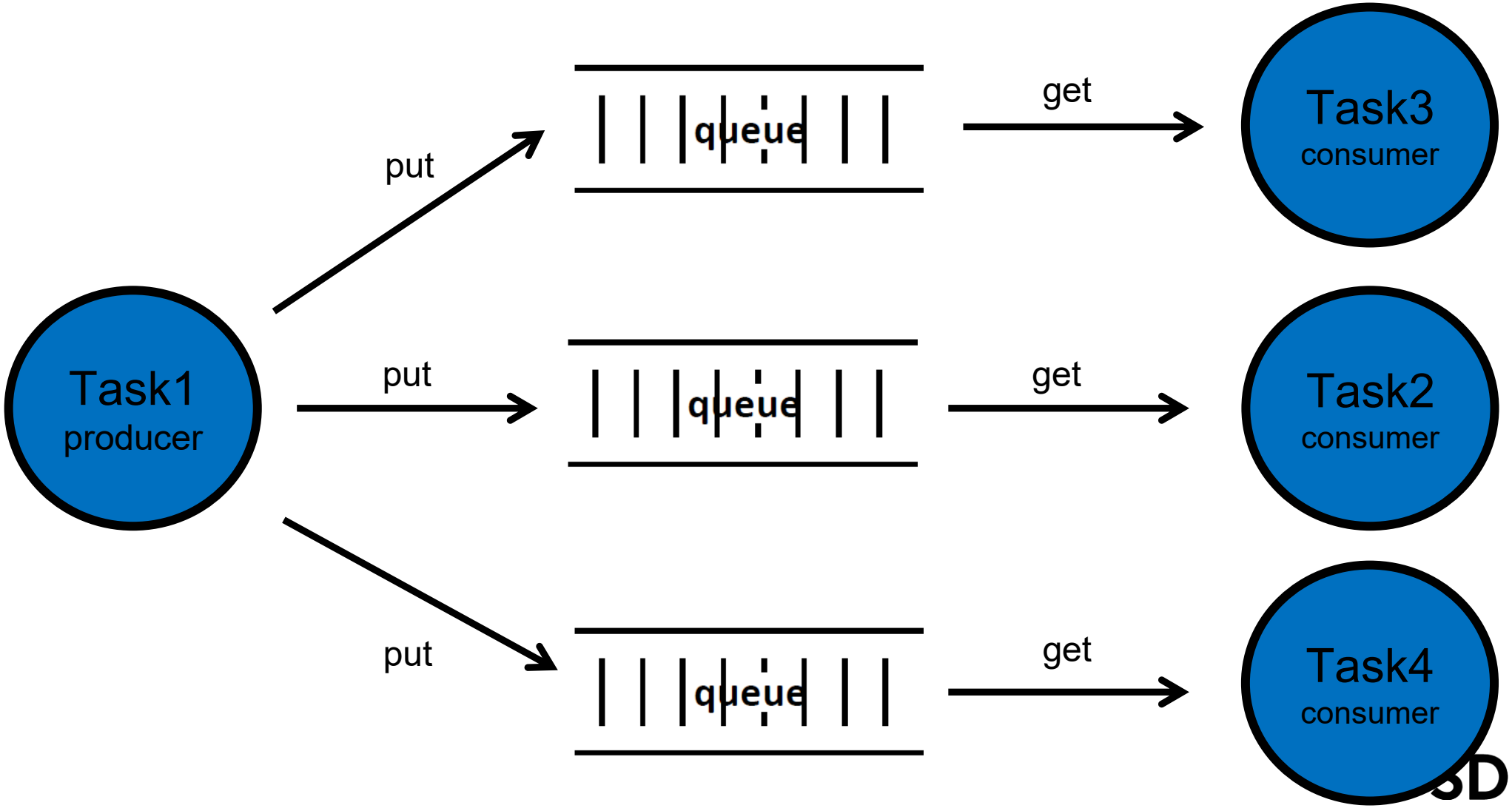
One consumer gets data from more queues



SDU

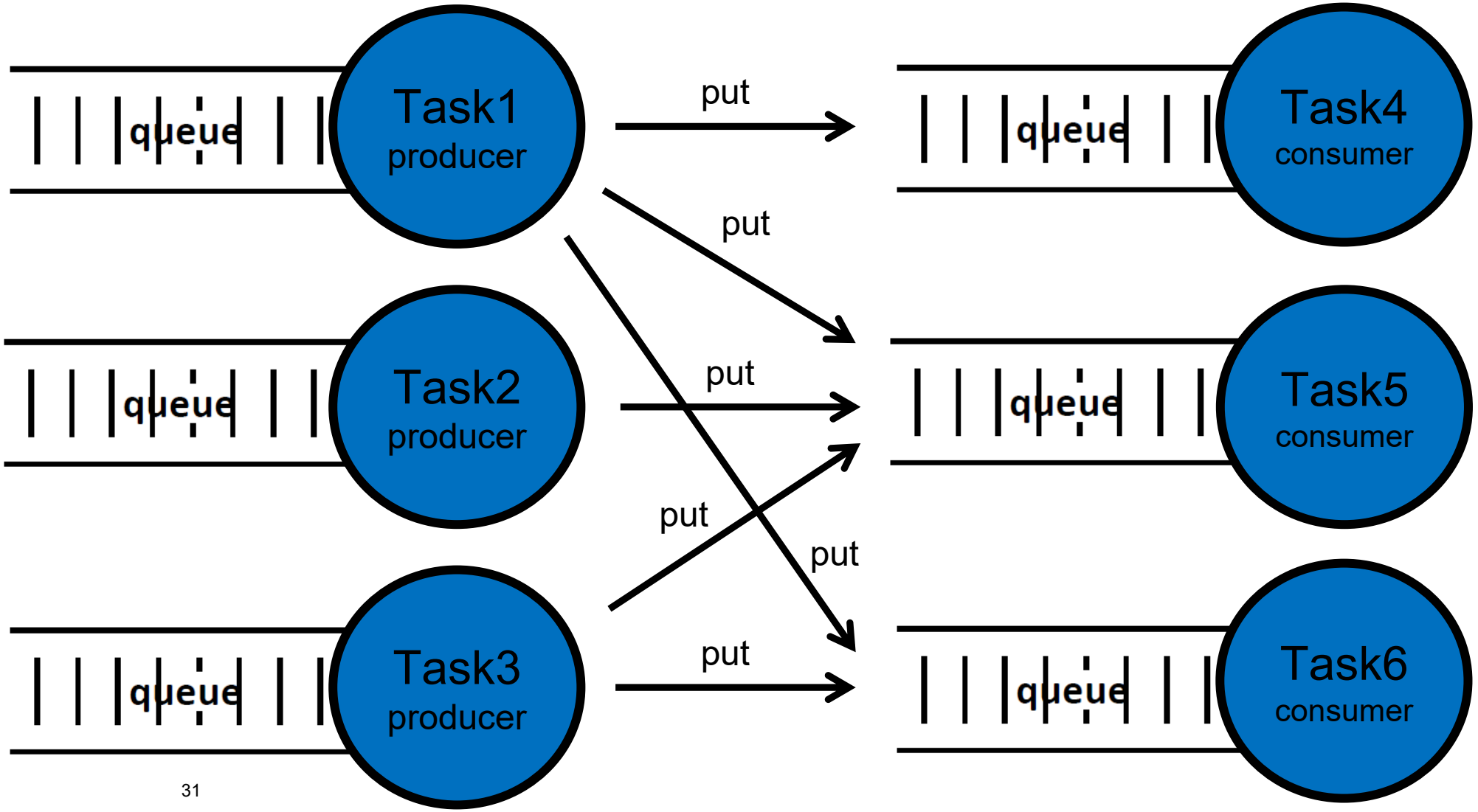
Queue topology

One producer sends data to more queues



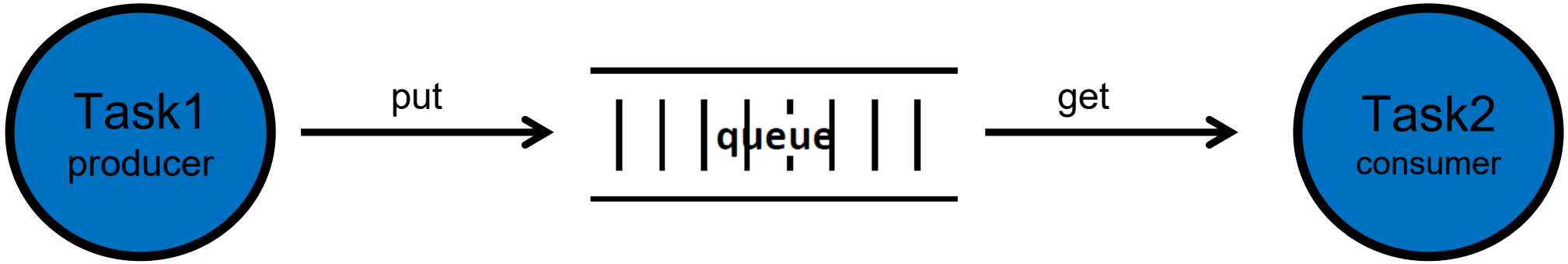
Queue topology

Some operating systems implement an input queue to each task



SDU

Queues in FreeRTOS



- Queue API
  - xQueueCreate();
  - xQueueSendToBack();
  - xQueueReceive();
  - uxQueueMessagesWaiting();
  - xQueuePeek();
- Queue ID
  - xQueueHandle

SDU

Exercise

- Can you write deadlocking tasks in RTCS? Why?
- Can 2 or 3 tasks that deadlock each other in FreeRTOS?
- How do you resolve such a deadlock?

SDU

SDU

Reentrancy

Reentrance

- A function is reentrant or pure if, while it is being executed, can be called again by itself or by another function
- …and it works every time
- i.e. more than one instance of a function can run at the same time

SDU

Errors that occur due to missing reentrance are hard to debug because…

- They seldom show up
- They can provide different faults
- They are dependent on memory configurations

SDU

A function is reentrant if…

- All variables are used atomic or are allocated to the specific instance of the function (not static)
- Common resources like hardware and global variables are used atomic or are protected by task synchronization elements (semaphores)
- It never changes its own code
- It never calls functions which are not reentrant

SDU

Atomic operations

- Ones that cannot be interrupted

Atomic operations

- Statements in C are not necessarily atomic but are typically executed as several assembler instructions

```
Foo++; // becomes
mov ax,[foo]
inc ax
mov [foo],ax;
```

SDU

Atomic operations

- Turn pieces of code atomic by wrapping it in ”interrupt disable”
- disabling interrupts reduces the real-time performance of the system (interrupt latency)
- use instead the known task synchronization methods (e.g. semaphores)

Non reentrant code

```
int foo;
void some_function( void )
{
     foo++;
}
```

SDU

Reentrant code

```
void some_function( void )
{
      int foo;
      foo++;
}
```

SDU

Static variables a problem for reentrance

```
void some_function( void )
{
     static int foo=1;
     foo++;
}
```

SDU

Common resources

- hardware and global variables must be protected by the known task synchronization methods:
  - Semaphores
  - Monitors (abstract data types which allows only one process to execute in a critical section at a time)

Common resources

```
void ChangeIP( unsigned long IP )
{
     Enter( semaphoreIP );
     IP[0] = Byte1(IP);
     IP[1] = Byte2(IP);
     IP[2] = Byte3(IP);
     IP[3] = Byte4(IP);
     Signal( semaphoreIP );
}
```

Avoid calling non-reentrant functions

- Check “old” code
- Beware of 3rd party libraries

Recursion

- a function is recursive, if it calls itself
- a recursive function must necessarily be reentrant
- A reentrant function must not necessarily be recursive

```
double power( double x, int exp )
{
     if( exp <= 0 )
          return(1);
     return( x*power(x, exp-1));
}
```

Writing reentrant code

- Easy to write
- Hard to retrofit (make non-reentrant code reentrant)

Read more about reentrancy

- http://www.ganssle.com/articles/areentra.htm

SDU